RLBWT-Based LCP Computation in Compressed Space for Terabase-Scale Pangenome Analysis

Mario Hernandez, Humberto Ortiz
Seminario Subgraduado II UPR, RP

Outline

1. Título
2. Pregunta investigativa
3. Problema
4. Usos principales
5. Datos genómicos
6. Compresión (overview)
7. RLBWT (detallado)
8. Construcción de índices (SA + LCP)
9. Problema de LCP
10. Comparacion entre metodo propuesto y previo
11. Solución (RLBWT-based)
12. Intuición del algoritmo
13. Métodos comparativos
14. Resultados (gráficas explicadas)
15. Impacto
16. Motivación personal
17. Conclusión
18. Referencias

Pregunta investigativa

¿Cómo pueden las estructuras de datos comprimidas reducir el uso de memoria en la indexación genómica a gran escala sin sacrificar el rendimiento?

Enfoque:

- Reducir memoria
- Reducir velocidad

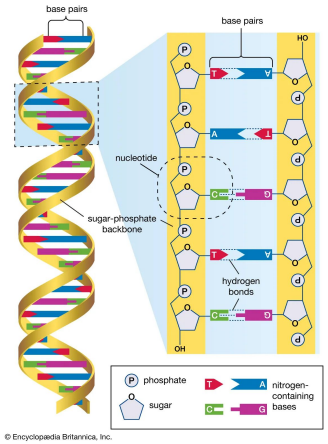
Datos Utilizados

- Secuencias de ADN (A, C, G, T)
- Tamaño: terabytes
- Alta repetición, compresión efectiva

	DNA Alignment	*Character Coding*
	000000000000000000011111111111111111111111	
	888888888999999999990000000000111111111122	
	123456789012345678901234567890123456789 01	1 2 3 4 5 6
Taxon 1	GCCTAGCCAAAGCTCTTCCAAGGTGACTCTCA**GTTCAA**GCT	2 0 3 2 0 4
Taxon 2	GCCTAGCCAAAGCTCTTCCAAGCTGACTCTCA------GCT	2 0 3 1 0 5
Taxon 3	GCCTAGCCTAAGCTCAACCAAGGTGTCTCTCA**GTTCAA**GCT	2 3 0 2 3 4
Taxon 4	GCCTAGCCTAAGCTCTTCCAAGGTGTCTCTCA**GTTCAA**GCT	2 3 3 2 3 4
Taxon 5	GCCTAGCCAAAGCTCTTCCAAGCTGACTCTCA------GCT	2 0 3 1 0 5
Taxon 6	CCCTAGCCAAAGCTCTTCCAAGCTGACTCTCA**GTTCAA**GCT	1 0 3 1 0 4
Taxon 7	CCCTAGCCAAAGCTCTTCCAAGCTGACTCTCA**GTTCAA**GCT	1 0 3 1 0 4
Taxon 8	GCCTAGCCTAAGCTCTTCCAAGCTGACTCTCA**GTTCAA**GCT	2 3 3 1 0 4

Usos principales

- Análisis de pangenomas
- Búsqueda de secuencias (pattern matching)
- Medicina personalizada
- Detección de enfermedades



Bottleneck de Memoria en Construccion de Indice Genomico

DNA datasets pueden ser de **petabytes**

Problema actual:

- Construcción de índices requiere mucha memoria
- Ejemplo: >2 TB RAM en datasets reales

No escala bien con el crecimiento de datos

Problema técnico: construcción de índices

Un índice permite buscar patrones en DNA rápidamente

Suffix Array (SA)

- Cada entrada apunta a un sufijo del texto original
- Tiempo de busqueda binaria: O(n * log n)

i	SA[i]	Suffix
0	6	$
1	5	A$
2	3	ANA$
3	1	ANANA$
4	0	BANANA$
5	4	NA$
6	2	NANA$

Longest Common Prefix (LCP)

- LCP[i] = longitud del prefijo común entre sufijos consecutivos en SA
- Evita comparar desde cero cada vez

Problema clave: No escala…

¿Por qué se usan juntos?

- SA, organiza los sufijos
- LCP, evita trabajo repetido

SA + LCP requieren **O(n) memoria**

- En datasets genómicos (TBs):
  - No escala

Métodos de compresión

Run-Length Encoding (RLE)

- Comprime repeticiones consecutivas
- Ej: AAAAA → (A,5)

BW Transform (BWT)

- Reorganiza el texto para agrupar caracteres similares
- No comprime directamente, pero prepara el texto para compresión eficiente

RLBWT (combinación)

- Datos genómicos son altamente repetitivos
- BWT agrupa esas repeticiones
- RLE las comprime eficientemente

Run-Length BW Transform (RLBWT) Explicado

- Guarda:
  - (char, run_length)
- Ejemplo:
  - BWT: AAAAATTTGG -> (A,5)(T,3)(G,2)

Comprime de O(n) a O(r), (n seq length, r # runs)

Cómo SA, ISA, LCP, BWT interactúan

Antes

Texto

- construir SA completo (ordenar sufijos)
- construir LCP completo (comparaciones entre sufijos)
- derivar información necesaria (thresholds / mínimos)

Despues (TeraLCP)

Texto

- construir BWT a RLBWT (runs)
- recorrer con LF-mapping
- usar ISA sampleado para ubicar posiciones
- inferir LCP solo en puntos necesarios (mínimos por run)

Comparando tiempo y memoria

Table 2: Minimum LCP per run computation

Dataset	Tool	Time (s)	Memory (MiB)
mtb152	lg + rlbwt2lcp	444	919
	pfp-thresholds	164	715
	lg + TeraLCP	**22**	**260**
chr19.1000	lg + rlbwt2lcp	55,893	77,820
	pfp-thresholds	10,334	41,631
	lg + TeraLCP	**1,918**	**8,601**

Table 3: Peak Memory Usage (GiB) for LCP Summary Tools

Dataset	rlbwt2lcp	pfp-thresholds	TeraLCP
mtb152	0.92	0.70	**0.16**
chr19.1000	76.00	40.66	**2.83**
human100	(est.) 460.18	791.92	**137.01**
human472	(est.) 1,947.93	2134.90	**172.16**
CommonBacteria	(est.) 9,901.64	-	**613.65**

Solución propuesta: indexación en espacio comprimido

Construir índices directamente sobre datos comprimidos usando **RLBWT**

Mejora:

- Memoria O(n) (SA & LCP completos)

  -> O(r) (r << n)

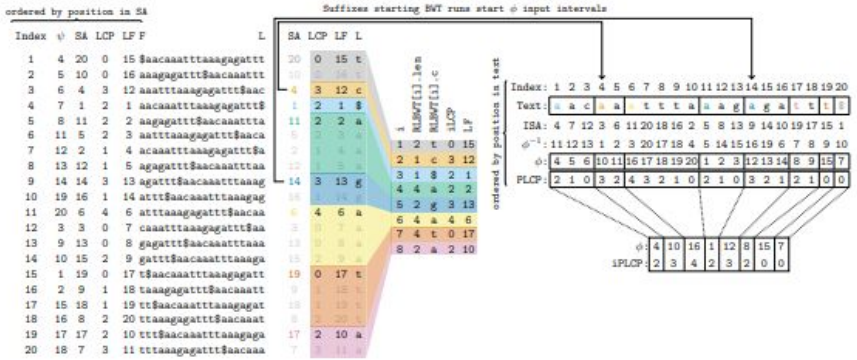


Figure 1: Relationships between uncompressed and run-length compressed *BWT* data structures

Idea intuitiva

Como funciona?

- No guarda todo el dataset
- Guarda
  - Indices claves
  - Estructuras compactas

Tecnicas

- Sampling de indices Importantes
- Navegacion con est. comprimidas
- Reconstruccion on-demand

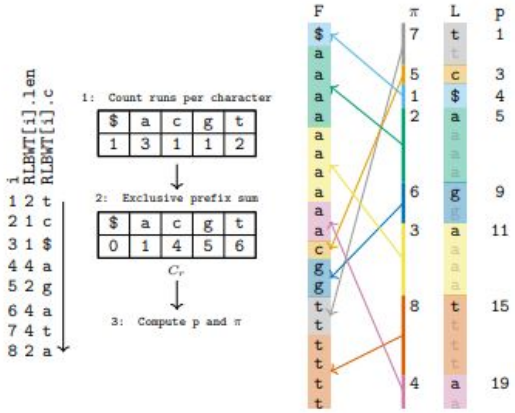


Figure 2: Construction of LF disjoint interval permutation in $O(r)$ space given an RLBWT. $T = aacaaatttaaagagattt\$$. The expanded view on the right is for illustrative purposes, it is not present in memory. L represents the BWT of T, $L = BWT(T)$.

Resultados

- 12.6x menos memoria
- 20-30x mas rapido

Escala datasets de:

- Terabytes
- Terabases (>1Tb)

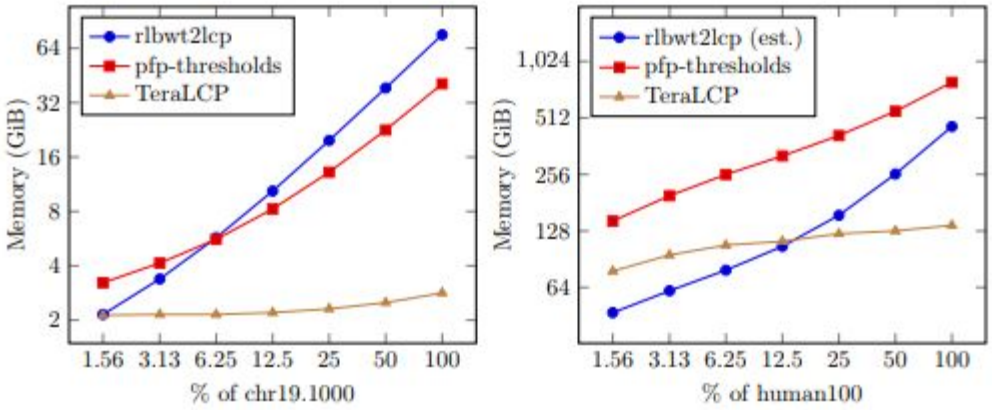


Figure 5: Memory scaling of TeraLCP, rlbwt2lcp, and pfp-thresholds

Conclucion

Los datos comprimidos:

- Reducen significativamente la memoria
- Mantienen eficiencia computacional

Respuesta a pregunta investigativa:

- Si, es posible indexar datasets masivos eficientemente usando estructuras comprimidas

Preguntas?

…

Referencias

- **Sanaullah, Ahsan, et al. “RLBWT-Based LCP Computation in Compressed Space for Terabase-Scale Pangenome Analysis.” (2026)**
- Gagie, Travis, Gonzalo Navarro, and Nicola Prezza. “Fully Functional Suffix Trees and Optimal Text Searching in BWT-Runs Bounded Space.” J. ACM, 2020.
- Nishimoto, Takaaki, and Yasuo Tabei. “Optimal-Time Queries on BWT-Runs Compressed Indexes.” ICALP, 2021.
- Boucher, Christina, et al. “Prefix-Free Parsing for Building Big BWTs.” Algorithms for Molecular Biology, 2019.
- Fischer, Johannes, and Enno Ohlebusch. “A Taxonomy of LCP-Array Construction Algorithms.” 2025.
- Kasai, Toru, et al. “Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications.” 2001.